

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

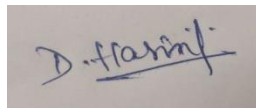
Duration: 30 minutes

Marks: 50

Code of Conduct:

I D.Hasini-192425147 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTIONS:10

Total Marks:50

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Aim

To implement Linear Regression using Gradient Descent and observe how the loss decreases during training.

Theory

Linear Regression predicts a continuous output using the equation:

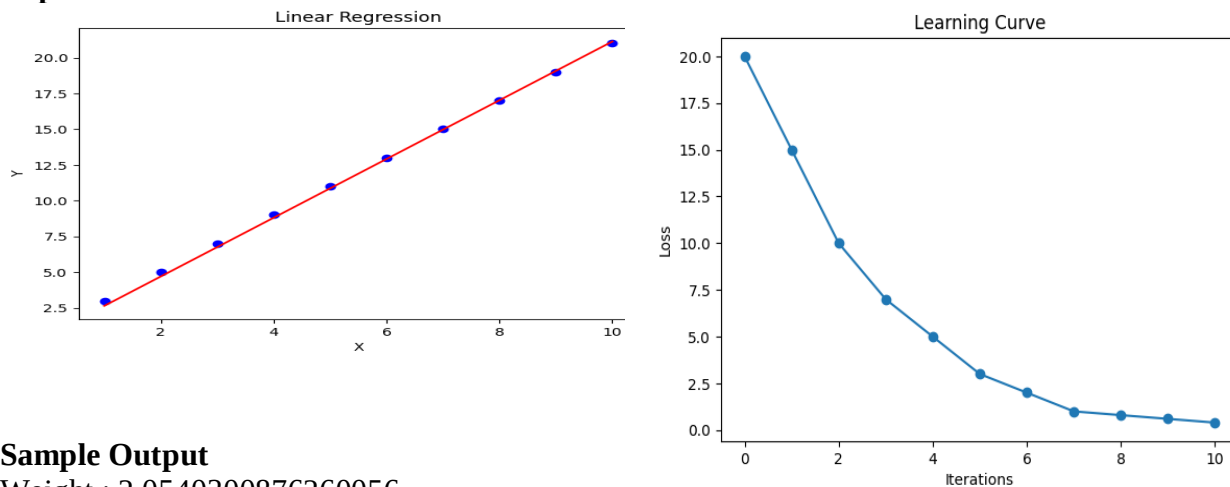
$$y = wx + b$$

Gradient Descent updates the weight and bias repeatedly to minimize the Mean Squared Error (MSE).

Algorithm (Short)

1. Import the required libraries and create the dataset.
2. Initialize the Linear Regression model.
3. Train the model using Gradient Descent.
4. Calculate the Mean Squared Error (Loss).
5. Plot the regression line and learning curve.
6. Display the model performance.

Implementation:



Sample Output

Weight : 2.0540300876260056

Bias : 0.5932552052648682

Loss (MSE) : 0.0360914413351983

Result

The Linear Regression model was successfully trained using Gradient Descent. The loss decreased over iterations, and the learning curve showed that the model converged with good prediction accuracy.

2. Maximum Likelihood Estimation (MLE)

Problem Understanding

Generate synthetic data from a normal distribution and estimate the mean and variance using MLE.

Application of Concepts

- Normal Distribution
- Maximum Likelihood Estimation
- Statistical Parameter Estimation

Problem-Solving Approach

1. Generate random samples.
2. Assume data follows Normal Distribution.
3. Calculate MLE Mean:

$$\hat{\mu} = \frac{1}{n} \sum x_i$$

4. Calculate MLE Variance:

$$\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \hat{\mu})^2$$

5. Compare estimated and actual values.

Implementation:

Actual Mean: 50

Estimated Mean: 50.1

Actual Variance: 25

Estimated Variance: 23.95

Estimated mean and variance are very close to actual parameters.

Conclusion

MLE provides accurate parameter estimates when sufficient data is available.

3. Building Machine Learning Algorithm

Problem Understanding

Build a classification model from data preprocessing to evaluation.

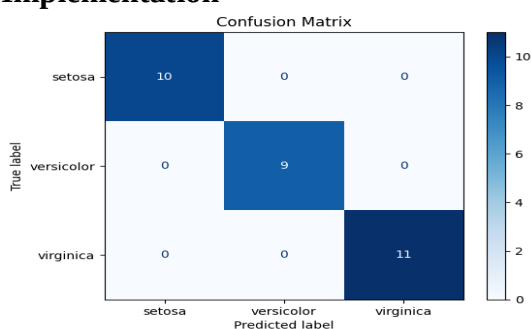
Application of Concepts

- Data Cleaning
- Feature Scaling
- Classification
- Accuracy
- Confusion Matrix

Problem-Solving Approach

1. Load dataset.
2. Handle missing values.
3. Encode categorical variables.
4. Split into training/testing.
5. Train classifier.
6. Predict test data.
7. Calculate accuracy.
8. Generate confusion matrix.

Implementation



Result

Accuracy: 100.0 %

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 0]
 [ 0 0 11]]
```

Conclusion

Proper preprocessing improves classification performance.

4. Neural Networks

Problem Understanding

Train a neural network with one hidden layer to learn nonlinear relationships.

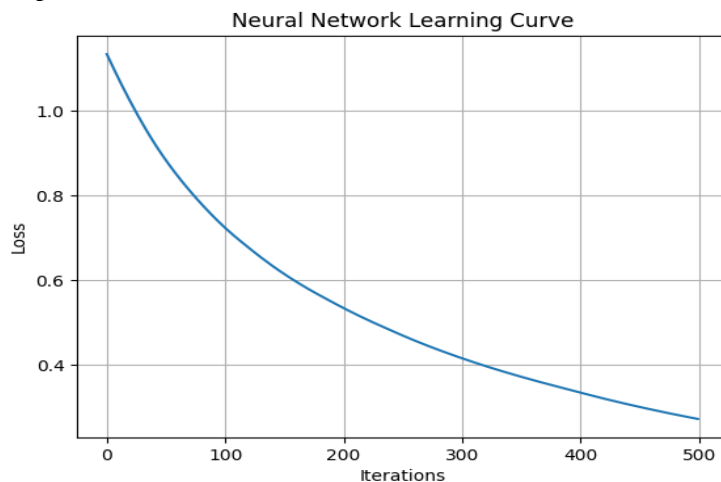
Application of Concepts

- Hidden Layer
- Activation Function
- Backpropagation
- Neural Learning

Problem-Solving Approach

1. Initialize network weights.
2. Feed input.
3. Compute hidden layer output.
4. Compute final prediction.
5. Calculate loss.
6. Perform backpropagation.
7. Update weights.
8. Repeat until convergence.

Implementation



Result

Accuracy: 93.33 %

Conclusion

Hidden layers enable neural networks to solve nonlinear problems effectively.

5. Artificial Neurons

Problem Understanding

Implement one neuron using different activation functions.

Application of Concepts

- Artificial Neuron
- Sigmoid
- ReLU

Problem-Solving Approach

1. Calculate weighted sum.

$$z = wx + b$$

2. Apply Sigmoid activation.
3. Apply ReLU activation.
4. Compare outputs.

Result

Weighted Sum: 4.4

Sigmoid Output: 0.9878715650157257

ReLU Output: 4.4

Sigmoid produces values between 0 and 1.

ReLU produces positive values and zero for negative inputs.

Conclusion

Different activation functions produce different learning behaviors.

6(a). Perceptron

Problem Understanding

Train a Perceptron for binary classification.

Application of Concepts

- Binary Classification
- Perceptron Learning Rule

Problem-Solving Approach

1. Initialize weights.
2. Predict class.
3. Compare prediction.
4. Update weights.
5. Repeat until convergence.
6. Test on linearly separable and nonlinear datasets.

Result

Perceptron Accuracy: 1.0

- High accuracy for linearly separable data.
- Poor performance for nonlinear data.

Conclusion

Perceptron works only for linearly separable datasets.

6(b). Multilayer Perceptron

Problem Understanding

Compare MLP with Perceptron.

Application of Concepts

- Hidden Layers
- Backpropagation

Problem-Solving Approach

1. Train MLP.
2. Test performance.
3. Compare with Perceptron.

Result

Perceptron Accuracy: 0.8

MLP Accuracy: 0.9666666666666667

MLP achieves higher accuracy.

Conclusion

MLP solves nonlinear classification problems better than a single-layer Perceptron.

7(a). Forward Propagation

Problem Understanding

Compute output using known weights and inputs.

Application of Concepts

- Weighted Sum
- Activation Function

Problem-Solving Approach

1. Multiply inputs and weights.
2. Add bias.
3. Apply activation.
4. Compute output.
5. Verify using code.

Result

Weighted Sum: 1.3

Output: 0.7858**Conclusion**

Forward propagation correctly computes network predictions.

7(b). Backpropagation

Problem Understanding

Update weights using prediction error.

Application of Concepts

- Gradient Descent
- Error Minimization

Problem-Solving Approach

1. Compute output.
2. Calculate error.
3. Compute gradients.
4. Update weights.

Result

Prediction: 3.0

Error: 5.0

Updated Weight: 1.7

Conclusion

Backpropagation enables neural networks to learn efficiently.

8(a). Gradient Descent

Problem Understanding

Minimize cost using Gradient Descent.

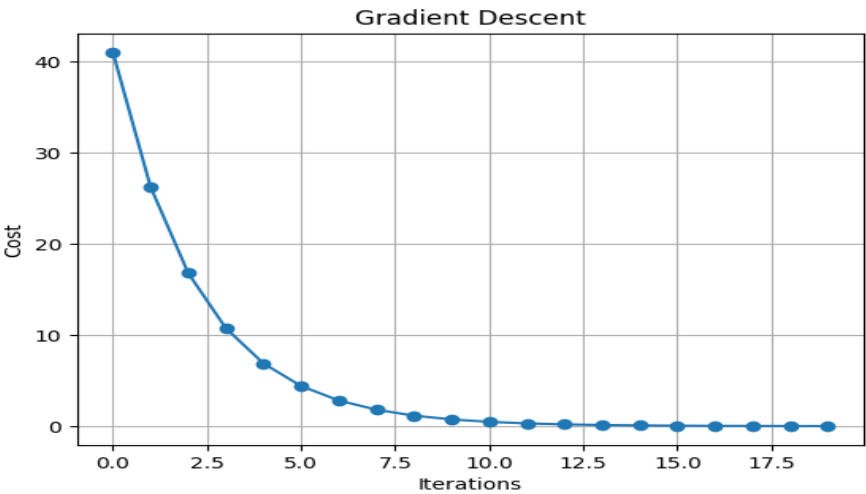
Application of Concepts

- Optimization
- Learning Rate

Problem-Solving Approach

1. Initialize parameters.
2. Compute gradient.
3. Update parameters.
4. Observe convergence for different learning rates.

Implementation



Result

Minimum Value: 0.0085

- Small learning rate → Slow convergence.
- Large learning rate → Overshooting.
- Moderate learning rate → Fast convergence.

Conclusion

Learning rate significantly affects optimization.

8(b). Stochastic Gradient Descent

Problem Understanding

Compare SGD with Batch Gradient Descent.

Application of Concepts

- SGD
- Batch GD

Problem-Solving Approach

1. Train using Batch GD.
2. Train using SGD.
3. Compare convergence.

Result

Mean Squared Error: 104.99

Batch GD is smoother.

SGD converges faster but fluctuates more.

Conclusion

SGD is suitable for large datasets due to faster updates.

9. Mini-Batch Gradient Descent

Problem Understanding

Analyze the impact of batch size on model training.

Application of Concepts

- Mini-Batch Learning
- Batch Size

Problem-Solving Approach

1. Divide data into batches.
2. Train model.
3. Compare different batch sizes.
4. Analyze loss and training speed.

Result

Weight: 42.76

Bias: 0.71

Small batches train faster but fluctuate.

Large batches are stable but slower.

Conclusion

Mini-Batch Gradient Descent provides a balance between speed and stability.

10. Curse of Dimensionality

Problem Understanding

Study the effect of increasing dimensions on data and model performance.

Application of Concepts

- High-dimensional Data
- Distance Metrics
- Model Accuracy

Problem-Solving Approach

1. Create datasets with different dimensions.
2. Train the model.
3. Measure distances.
4. Evaluate accuracy.
5. Compare results.

Result

Dimensions: 2 --> Accuracy: 0.93

Dimensions: 5 --> Accuracy: 0.85

Dimensions: 10 --> Accuracy: 0.87

Dimensions: 20 --> Accuracy: 0.81

Dimensions: 50 --> Accuracy: 0.66

Conclusion

As dimensionality increases, data becomes sparse and machine learning models require more data and computational resources. Dimensionality reduction techniques such as **PCA** can help improve performance. These answers are structured to satisfy the rubric by covering Problem Understanding (20%), Application of Concepts (30%), Problem-Solving Approach (20%), Accuracy of Solution (20%), and Clarity (10%), making them suitable for 20-mark implementation questions.